

LLMProvider

LLMProvider

Une interface, 43 fournisseurs — avec disjoncteurs, réessais et surveillance de santé intégrés.

Résumé

LLMProvider est un module Go générique et réutilisable qui définit une interface unifiée LLMProvider ainsi que les mécanismes de résilience en production qui l'accompagnent — disjoncteur, surveillance de santé, réessai avec backoff, chargement différé — et propose 43 implémentations concrètes de fournisseurs derrière ce contrat unique, avec un adaptateur générique compatible OpenAI et une découverte de modèles honnête, sans recours à des solutions de repli codées en dur.

Description courte

Un module Go réutilisable exposant une interface LLMProvider (Complete, CompleteStream, HealthCheck, GetCapabilities, ValidateConfig) ainsi que des primitives de tolérance aux pannes — disjoncteur, moniteur de santé, réessai avec backoff et jitter, initialisation différée — sur 43 adaptateurs de fournisseurs et un adaptateur générique compatible OpenAI. Thread-safe.

Description détaillée

LLMProvider est la couche d'abstraction dont tout service consommant LLM a besoin, mais que presque personne ne construit correctement — cette plomberie ingrate qui fait la différence entre une démonstration et un système capable de résister au trafic réel. Il définit une interface unique, consciente des capacités — Complete, CompleteStream, HealthCheck, GetCapabilities, ValidateConfig — afin que le code applicatif cible un seul contrat, quel que soit le fournisseur parmi les 43 qui répond à l'appel. Il intègre ensuite les mécanismes de

durcissement opérationnel qui transforment des appels fragiles à des fournisseurs en quelque chose que l'on peut exécuter en production sans retenir son souffle.

Un disjoncteur à trois états (fermé → ouvert → semi-ouvert) enveloppe de manière transparente n'importe quel fournisseur — *y compris son canal de streaming*, où un flux vide est correctement comptabilisé comme un échec — afin qu'un backend défaillant puisse déclencher l'ouverture du disjoncteur et éviter de faire tomber tout le service avec lui. Un `CircuitBreakerManager` central suit tous les disjoncteurs simultanément. Un moniteur de santé configurable évalue en continu les fournisseurs selon les états *sain*, *dégradé*, *hors service* ou *inconnu*, en fonction de seuils et d'intervalles prédéfinis, permettant ainsi de détecter une dégradation avant qu'elle ne se transforme en panne. La logique de réessai superpose un backoff exponentiel avec du jitter, prenant des décisions en fonction du statut — réessayer les erreurs pertinentes (429, 5xx, pannes réseau transitoires), sans gaspiller de cycles sur les erreurs 4xx ou un contexte annulé — tout en limitant les délais pour éviter une tempête de backoff. Enfin, un motif d'initialisation différée retarde la construction de chaque fournisseur jusqu'à sa première utilisation réelle, une conception délibérée qui rend l'enregistrement des 43 fournisseurs pratiquement gratuit.

Le module inclut 43 paquets de fournisseurs concrets ainsi qu'un adaptateur *generic* compatible OpenAI, qui implémente l'interface complète pour *n'importe quel* endpoint `/v1/chat/completions` — authentification Bearer, streaming SSE avec gestion correcte du signal `[DONE]` — permettant ainsi à un fournisseur sans paquet dédié de devenir un citoyen de première classe dès que l'adaptateur est pointé vers son URL. Les identifiants sont résolus en un seul endroit (`apikeys`, en suivant une convention stricte `ApiKey_<Fournisseur>`), éliminant ainsi toute une classe de bugs du type « *la clé codée en dur passe les tests, mais la vraie clé n'a jamais été configurée, et le produit plante en production* ». La découverte des modèles est délibérément, presque obstinément honnête : elle interroge les API des fournisseurs en direct derrière un cache TTL, et — conformément aux règles de gouvernance — l'ancien niveau de repli codé en dur a été purement et simplement supprimé. En cas d'échec de la découverte en direct, `LLMProvider` ne retourne *rien* plutôt qu'un catalogue obsolète, de sorte qu'un appelant ne se voit jamais attribuer un identifiant de modèle qui semble valide mais ne peut pas être invoqué. Chaque composant est conçu pour être thread-safe et supporter une utilisation concurrente.

Contenu

Pourquoi nous l'avons conçu

Les appels naïfs à LLM échouent en production — les fournisseurs limitent les requêtes, dégradent leurs services ou tombent en panne, et un seul backend défaillant peut entraîner tout un service dans sa chute. Les catalogues de modèles dérivent, et les listes codées en dur transmettent aux appelants des identifiants qui ne fonctionnent plus. LLMProvider centralise l'interface, les mécanismes de résilience et une découverte honnête, de sorte que chaque consommateur hérite gratuitement de la tolérance aux pannes et de la fiabilité.

Pourquoi c'est révolutionnaire

Il réduit l'intégration d'un fournisseur LLM à une seule étape — implémenter une interface unique, ou simplement pointer l'adaptateur générique vers un endpoint — puis enveloppe ce fournisseur, de manière automatique et transparente, dans un système de *circuit breaking*, de surveillance de l'état de santé et de réessais avec *backoff* et *jitter*. La résilience cesse d'être quelque chose que chaque équipe réinvente (mal, sous la pression des délais, après la première panne) pour devenir le comportement par défaut de la bibliothèque, appliqué à l'ensemble des 43 backends. L'ingénierie de la fiabilité est écrite une fois, rigoureusement testée, et héritée gratuitement par tous ceux qui l'importent.

Ce qui est innovant

- **Une interface unique consciente des capacités** — achèvement, *streaming*, santé, capacités et validation de configuration regroupés dans un seul contrat que chaque backend respecte à l'identique.
- **Enveloppement transparent par un *circuit breaker* — y compris pour les flux.** Le *breaker* protège le canal de `CompleteStream`, et pas seulement les requêtes/réponses, et traite un flux vide comme l'échec qu'il représente réellement — avec une notification des auditeurs sûre contre les blocages, hors verrouillage.
- **43 modules dédiés à des fournisseurs + un adaptateur générique compatible OpenAI** — les modules dédiés restent légers, et tout fournisseur non répertorié parlant `/v1/chat/completions` fonctionne dès que vous pointez l'adaptateur vers lui.
- **Une seule autorité de gestion des identifiants (apikeys)** — un seul endroit lit les variables d'environnement `ApiKey_<Fournisseur>`, éliminant structurellement l'écart « tests verts, produit cassé » au lieu de se contenter de le signaler.

- **Découverte honnête des modèles (pas de repli codé en dur)** — les API des fournisseurs en direct, derrière un cache à durée de vie limitée ; en cas d'échec, elle retourne nil, jamais un catalogue obsolète ou fabriqué qui distribue des identifiants inutilisables.
- **Initialisation paresseuse avec `sync.Once`** — la construction est différée jusqu'à la première utilisation, de sorte que l'enregistrement des 43 fournisseurs ne coûte presque rien tant que vous n'en appelez pas un.
- **Pile de défis anti-blanchiment, multilingue** — un véritable *runner* qui teste le comportement des *circuits*, de la santé et des réessais dans cinq localisations, contrôlé par des tests de mutation appariés (le code non muté doit sortir avec un code 0 ; une mutation injectée doit forcer une sortie 99), de sorte qu'une suite réussie garantit un comportement fonctionnel.

Principaux défis techniques et solutions apportées

- **Défaillances en cascade des fournisseurs.** Un backend instable ne doit pas entraîner tout un service dans sa chute. Solution : un *circuit breaker* à trois états (fermé → ouvert → semi-ouvert) qui enveloppe de manière transparente tout fournisseur *et son flux*, bascule en mode ouvert en cas d'échecs répétés, sonde la reprise en mode semi-ouvert, et est coordonné de manière centralisée par un `CircuitBreakerManager`.
- **Erreurs transitoires et limites de débit.** Solution : un *backoff* exponentiel sensible au statut, avec *jitter* — $\text{min}(\text{DelaiInitial} \cdot \text{Multiplicateur}^{(n-1)}, \text{DelaiMax}) \pm \text{jitter}$ — de sorte que les réessais s'étalent au lieu de se synchroniser en un effet de troupeau. Il ne réessaie que ce qui doit l'être (429, 500, 502, 503, 504 et erreurs réseau) et refuse de gaspiller des tentatives sur un contexte annulé ou tout autre code 4xx.
- **Mise à l'échelle avec de nombreux fournisseurs enregistrés mais inutilisés.** Avec 43 fournisseurs enregistrés mais seulement quelques-uns actifs dans un service donné, une construction impatiente serait un pur gaspillage. Solution : initialisation paresseuse protégée par `sync.Once`, de sorte que seuls les fournisseurs effectivement appelés paient leur coût de configuration.
- **Distribution d'identifiants de modèles invalides.** Solution : suppression pure et simple du niveau de repli codé en dur pour la découverte (conformément à CONST-036) et retour de nil en cas d'échec de la découverte en direct — avec une copie défensive à la restitution pour empêcher un appelant de modifier le cache ou d'entrer en conflit avec un autre lecteur. La fiabilité est imposée structurellement, et non par convention.

- **Correction du *streaming* et de la concurrence.** Le mode de défaillance subtil est un blocage entre le verrou du *breaker* et ses rappels d'auditeurs. Solution : capture instantanée des auditeurs et notification hors verrouillage sous un délai de 5 secondes, avec déverrouillage avant notification lors de la réinitialisation — chaque composant étant conçu pour une utilisation concurrente et validé par la suite - race.

Contenu

Pile technologique

- **Go (1.25.3)** — choisi pour sa gestion de premier ordre de la concurrence, ses binaires statiques et une bibliothèque standard robuste ; il intègre le module, l'interface, toutes les primitives de résilience et les 43 adaptateurs.
- **net/http (stdlib)** — HTTP délibérément sans dépendance : alimente les clients par fournisseur, l'adaptateur générique compatible OpenAI et les appels de découverte en direct, éliminant ainsi tout transport tiers à auditer ou à corriger.
- **logrus** — journalisation structurée et sensible aux niveaux, précisément là où les opérateurs ont besoin de visibilité : dans les transitions d'état du disjoncteur et le chemin de découverte.
- **testify** — pilote la suite de tests et, surtout, le verrouillage des branches de mutation qui donne un sens à une exécution réussie.
- **yaml.v3** — analyse les bundles d'internationalisation et la configuration dans un format restant modifiable par l'humain.
- **digital.vasic.models** — les types partagés LLMRequest / LLMResponse / ProviderCapabilities, centralisés pour que chaque adaptateur utilise le même vocabulaire (une dépendance runtime documentée).
- **Paquets propriétaires** — circuit, health, retry, apikeys, discovery, providers/ (43 fournisseurs + generic), et i18n : la surface de résilience et d'intégration est divisée en petites unités testables indépendamment, plutôt qu'en un monolithe.
- **.env + ~/api_keys.sh (convention ApiKey_<Provider>)** — une source unique et sans ambiguïté pour les identifiants, permettant un câblage identique des clés en tests et en production.
- **Suite de tests de concurrence Makefile (-race -p 1) + Challenge runner** — l'épine dorsale anti-bidon : le détecteur de courses prouve la correction de la concurrence, et le Challenge runner soumet le comportement réel à des scénarios de chaos, ddos, de mise à l'échelle, de stress, de découverte en direct et sans suspension.

Statut et notes de transparence

- **Statut : bêta.** Un module réutilisable découplé ; le dépôt GitHub est public.
- **Licence : à déterminer.** Incohérente — `doc.go` indique MIT tandis qu'un fichier LICENSE de type Apache-2.0 est présent — à vérifier avant publication.
- LLMsVerifier est la source unique de référence en amont pour le catalogue de modèles canoniques. Le manifeste `helix-deps.yaml` semble obsolète (déclare `deps: []` alors que la documentation mentionne une dépendance à `digital.vasic.models`) ; la « Tier 2 (models.dev) » de la découverte est une ébauche planifiée, non active.

Niveau de priorité : Helix-primaire (cluster LLM-infrastructure — module réutilisable découplé). Classé après HelixTrack.